Universidad de los Andes

Departamento de Ingeniería Industrial · Modelado de Sistemas bajo Incertidumbre

Lectura: Congruencia y Transformación

Alejandra Tabares Pozos

Índice

1. El Caso de Estudio: Auditoría al Casino Andino **2**

2. La Paradoja del Computador Determinístico **2**

3. El Motor Matemático: Aritmética de Reloj **2**

3.1. El Concepto de Residuo 3

3.2. Congruencia: La Igualdad en el Reloj 3

4. Ecuaciones Lineales en el Entorno Modular **3**

4.1. El Problema de la Inversión 4

5. El Generador Congruencial Lineal (LCG) **4**

5.1. Ejemplo Numérico: Auditoría En Cámara Lenta 5

5.2. Análisis del Resultado 6

6. La Traducción: Método de la Transformada Inversa **6**

6.1. El Concepto: Entrando por el Eje Y 6

6.2. Ejemplo Inicial: La Función Rampa 7

6.3. Ejemplo 1: Distribución Exponencial (Tiempos de Juego) 7

6.4. Ejemplo 2: Distribución Uniforme General (Rangos de Apuesta) 8

6.5. Resumen de la Sección 9

1. El Caso de Estudio: Auditoría al Casino Andino

Imagina que has sido contratado para una tarea crítica: auditar el motor de juego de Casino Andino, una nueva plataforma de apuestas en línea. Los usuarios han comenzado a sospechar. En los foros de internet, circulan rumores de que las máquinas tragamonedas son predecibles y que los grandes premios, matemáticamente posibles según la publicidad, nunca aparecen en la realidad.

Tu misión como ingeniero no es jugar, sino abrir la caja negra del software y responder a preguntas fundamentales que tocan la fibra misma de la computación:

- ¿Cómo puede una computadora, que es una máquina de lógica exacta (2+2 siempre es 4), generar suerte o azar?
- ¿Es posible que el generador de números aleatorios esté atascado en un ciclo que excluye los premios mayores?
- ¿Cómo se traduce un simple número generado por el procesador en un evento complejo como Tres Cerezas o un tiempo de espera?

Para realizar esta auditoría, debemos descender al nivel más bajo de la simulación: la aritmética de los números enteros y las transformaciones de probabilidad.

2. La Paradoja del Computador Determinístico

El obstáculo epistemológico central en la simulación es que el azar verdadero es ajeno a la naturaleza de un computador digital estándar. Un algoritmo es, por definición, una secuencia finita de instrucciones bien definidas. Si ejecutamos el mismo código con las mismas entradas, obtendremos invariablemente la misma salida.

Por lo tanto, en ingeniería de simulación, no buscamos azar verdadero, sino **Pseudo-Aleatoriedad**: secuencias numéricas generadas por una fórmula determinística que poseen propiedades estadísticas indistinguibles del azar real para el propósito del estudio.

El flujo de trabajo estándar en cualquier simulación moderna (desde el clima hasta las finanzas) sigue este esquema: 1. Generación: Producir un número U que se comporte como una distribución Uniforme entre 0 y 1 ($U \sim [0, 1)$). 2. Transformación: Convertir ese U en la variable de interés X (ej. tiempo de llegada, monto de apuesta).

Nuestro primer objetivo es entender el motor que produce U: la Aritmética Modular.

3. El Motor Matemático: Aritmética de Reloj

Para entender cómo las computadoras barajan números, debemos olvidar por un momento la recta numérica infinita y pensar en círculos.

3.1. El Concepto de Residuo

Imagina un reloj analógico de 12 horas. Si son las 10:00 y transcurren 5 horas, no decimos que son las 15:00, sino las 3:00. Matemáticamente, lo que hacemos es enrollar la recta numérica alrededor del círculo del reloj.

$$10 + 5 = 15$$

$$15 \div 12 = 1 \text{ (vuelta completa) con un residuo de } \mathbf{3}.$$

En la aritmética modular, el residuo es lo único que importa. Definimos la operación módulo (mód) como el acto de quedarse con el residuo de una división entera.

El Operador Módulo

Dados un entero a y un entero positivo m (el módulo), escribimos:

$$r = a \mod m$$

Donde r es el único entero tal que $0 \leq r < m$ y $a = qm + r$ para algún entero q.

3.2. Congruencia: La Igualdad en el Reloj

Decimos que dos números a y b son congruentes módulo m si, al dividirlos por m, dejan el mismo residuo. Esto se escribe:

$$a \equiv b \pmod{m}$$

Esto tiene una implicación práctica poderosa: en el mundo del módulo m, a y b son efectivamente el mismo número. Por ejemplo, en un reloj de tamaño 12 ($m = 12$):

$$15 \equiv 3 \pmod{12}$$

$$27 \equiv 3 \pmod{12}$$

Tanto el 15, el 27, como el 3, representan la misma posición en la esfera del reloj.

4. Ecuaciones Lineales en el Entorno Modular

Para diseñar un generador de números aleatorios, necesitamos manipular estos números. Surgen entonces ecuaciones del tipo:

$$ax \equiv b \pmod{m}$$

Esto se lee: ¿Por qué número x debo multiplicar a a, tal que al dar vueltas al reloj de tamaño m, termine en la posición b?.

4.1. El Problema de la Inversión

En álgebra normal, para resolver $5x = 10$, dividimos por 5. En aritmética modular, la división no existe como tal; en su lugar, buscamos un Inverso Multiplicativo.

Buscamos un número a^{-1} tal que:

$$a \cdot a^{-1} \equiv 1 \pmod{m}$$

Si encontramos este inverso, podemos despejar la x multiplicando ambos lados por a^{-1}. Sin embargo, aquí yace una trampa crítica para nuestro diseño de simulación: no todos los números tienen inverso modular.

Condición de Existencia del Inverso

El inverso de a módulo m existe si y solo si a y m son primos relativos, es decir, su Máximo Común Divisor es 1:

$$\gcd(a, m) = 1$$

Ejemplo de Auditoría: Supongamos que el algoritmo del casino usa un reloj de tamaño 12 ($m = 12$) y multiplica sus estados por 6 ($a = 6$). ¿Existe un estado x que nos lleve a la posición 1? ($6x \equiv 1 \pmod{12}$). Revisemos los múltiplos de 6 en este reloj: - $6 \times 1 = 6 \equiv 6$ - $6 \times 2 = 12 \equiv 0$ - $6 \times 3 = 18 \equiv 6$ - $6 \times 4 = 24 \equiv 0$

La secuencia oscila entre 0 y 6. Nunca toca el 1, ni el 2, ni el 3... Al elegir un multiplicador $a = 6$ que comparte factores con $m = 12$, hemos roto el reloj, haciéndolo incapaz de visitar todas las horas. Un generador de azar basado en esto sería defectuoso y fraudulento, pues nunca generaría ciertos resultados.

5. El Generador Congruencial Lineal (LCG)

Ahora uniremos las piezas. Hemos aprendido que la aritmética modular funciona como un reloj que siempre mantiene los números dentro de un rango fijo $\{0, 1, \ldots, m-1\}$. El Generador Congruencial Lineal (LCG) no es más que una regla para mover la aguja de ese reloj de forma que visite todas las horas posibles en un orden aparentemente desordenado.

El algoritmo conecta el pasado (Z_n) con el futuro (Z_{n+1}) mediante la ecuación:

$$Z_{n+1} = (a \cdot Z_n + c) \mod m$$

Aquí está la conexión crítica que faltaba: Z es la posición en el reloj. Pero nuestra simulación necesita un número entre 0 y 1 (una probabilidad). Por eso, siempre realizamos un segundo paso: la Normalización.

$$U_{n+1} = \frac{Z_{n+1}}{m}$$

Al dividir la posición del reloj entre su tamaño total, convertimos un entero grande en una fracción decimal. Si el reloj tiene tamaño m, los resultados posibles son $0/m, 1/m, 2/m, \ldots, (m-1)/m$.

5.1. Ejemplo Numérico: Auditoría En Cámara Lenta

Para visualizar esto, no usemos un computador con $m = 2,000,000,000$. Usemos un mini-computador de papel con un reloj de tamaño $m = 8$. Nuestros parámetros de diseño serán:

- **Módulo ($m = 8$):** El reloj tiene las posiciones $\{0, 1, 2, 3, 4, 5, 6, 7\}$.
- **Multiplicador ($a = 5$):** Un número elegido para desordenar la secuencia.
- **Incremento ($c = 1$):** El desplazamiento constante.
- **Semilla ($Z_0 = 0$):** Arrancamos con la aguja en el cero.

Ejecutemos el algoritmo a mano para ver cómo generamos el caos determinístico:

Iteración 1: 1. *Expansión:* Calculamos $aZ_0 + c = 5(0) + 1 = 1$. 2. *Módulo:* $1 \div 8 = 0$ con residuo **1**. El nuevo estado es $Z_1 = 1$. 3. *Normalización:* $U_1 = 1/8 = \mathbf{0.125}$.

Iteración 2: (Usamos $Z_1 = 1$ como entrada) 1. *Expansión:* $5(1) + 1 = 6$. 2. *Módulo:* $6 \div 8 = 0$ con residuo **6**. El nuevo estado es $Z_2 = 6$. 3. *Normalización:* $U_2 = 6/8 = \mathbf{0.750}$.

Iteración 3: (Usamos $Z_2 = 6$ como entrada) 1. *Expansión:* $5(6) + 1 = 31$. 2. *Módulo:* ¿Dónde cae el 31 en un reloj de 8?

$$31 = 3 \times 8 + 7$$

El residuo es **7**. El nuevo estado es $Z_3 = 7$. 3. *Normalización:* $U_3 = 7/8 = \mathbf{0.875}$.

Iteración 4: (Usamos $Z_3 = 7$ como entrada) 1. *Expansión:* $5(7) + 1 = 36$. 2. *Módulo:* $36 = 4 \times 8 + 4$. El residuo es **4**. Nuevo estado $Z_4 = 4$. 3. *Normalización:* $U_4 = 4/8 = \mathbf{0.500}$.

Tabla de Rastreo Completa

Si continuamos el proceso, la secuencia de estados enteros (Z) y sus salidas uniformes (U) es:

Paso (n)	**Cálculo** ($5Z+1$)	**Residuo** (Z)	**Salida** (U)
0	Semilla	0	-
1	1	**1**	0.125
2	6	**6**	0.750
3	31	**7**	0.875
4	36	**4**	0.500
5	21	**5**	0.625
6	26	**2**	0.250
7	11	**3**	0.375
8	16	**0**	0.000

5.2. Análisis del Resultado

Observemos la columna Residuo (Z). La secuencia generada fue:

$$0 \to 1 \to 6 \to 7 \to 4 \to 5 \to 2 \to 3 \to 0 \ldots$$

¡Hemos logrado el objetivo! 1. Cobertura: El generador visitó **todos** los números posibles del 0 al 7 antes de volver al 0. No dejó huecos en el reloj. 2. Desorden: Los números no salieron en orden (0, 1, 2...), sino saltando de un lado a otro (de 0.125 saltó a 0.750). 3. Normalización: Aunque trabajamos con enteros, la columna U nos entregó decimales dispersos en el intervalo $[0, 1)$, que es exactamente lo que necesitamos para simular probabilidades.

En un computador real, m no es 8, sino un número gigante (como $2^{31}-1 \approx 2$ mil millones). Esto hace que los saltos entre los decimales sean tan minúsculos ($1/2^{31}$) que, para efectos prácticos, parecen una línea continua de números reales aleatorios. Pero la mecánica lógica es idéntica a la de nuestro reloj de 8 horas.

6. La Traducción: Método de la Transformada Inversa

Una vez que el Generador Congruencial Lineal (LCG) nos entrega un número uniforme $U \in [0, 1)$, tenemos materia prima estadística. Sin embargo, el casino no funciona con números entre 0 y 1; funciona con tiempos de espera, montos de apuestas y tipos de premios.

Necesitamos un mecanismo de traducción. La herramienta universal para esto es la Función de Distribución Acumulada (CDF), denotada como $F(x)$.

6.1. El Concepto: Entrando por el Eje Y

La función $F(x)$ responde a la pregunta: ¿Cuál es la probabilidad de obtener un valor menor o igual a x?. Como las probabilidades siempre van de 0 a 1, el rango de $F(x)$ coincide exactamente con el rango de nuestros números aleatorios U.

El Método de la Transformada Inversa consiste en revertir la pregunta:

- **Directo:** Dado un valor x, ¿cuál es su probabilidad acumulada U? ($U = F(x)$).
- **Inverso:** Dado una probabilidad acumulada U (generada por el computador), ¿qué valor x le corresponde? ($x = F^{-1}(U)$).

Visualmente, es como lanzar un dardo al eje Y (el número U), moverse horizontalmente hasta chocar con la curva de la función, y bajar verticalmente para leer el valor simulado en el eje X.

6.2. Ejemplo Inicial: La Función Rampa

Antes de usar distribuciones complejas, veamos la mecánica con una función inventada muy simple. Supongamos que una variable X tiene una densidad triangular simple definida en el rango $[0, 1]$:

$$f(x) = 2x \quad \text{para } 0 \leq x \leq 1$$

Paso 1: Hallar la Acumulada $F(x)$. Integramos la densidad:

$$F(x) = \int_0^x 2t\,dt = t^2\Big|_0^x = x^2$$

Sabemos que $F(x) = x^2$.

Paso 2: Igualar a U y Despejar. El computador nos da un número U (ej. 0.64). Igualamos:

$$U = x^2$$

Despejamos x:

$$x = \sqrt{U}$$

Resultado: Esta es nuestra **fórmula generadora**.

- Si el LCG genera $U = 0.64$, nuestra variable simulada es $x = \sqrt{0.64} = 0.8$.
- Si el LCG genera $U = 0.25$, nuestra variable simulada es $x = \sqrt{0.25} = 0.5$.

Al aplicar la raíz cuadrada a los números uniformes, transformamos su distribución: ahora es más probable obtener valores altos (cercanos a 1) que bajos, tal como dicta la densidad $f(x) = 2x$.

6.3. Ejemplo 1: Distribución Exponencial (Tiempos de Juego)

En el contexto del Casino Andino, el tiempo que un jugador pasa en una máquina suele modelarse con una distribución Exponencial con media β (o tasa $\lambda = 1/\beta$).

Paso 1: La Acumulada. La CDF de la exponencial es bien conocida:

$$F(x) = 1 - e^{-x/\beta} \quad \text{para } x \geq 0$$

Paso 2: Igualar a U.

$$U = 1 - e^{-x/\beta}$$

Paso 3: Despejar x.

$$e^{-x/\beta} = 1 - U$$

Aplicamos logaritmo natural a ambos lados:

$$-x/\beta = \ln(1 - U)$$

$$x = -\beta \ln(1 - U)$$

Nota de Optimización

Dado que U es una variable aleatoria uniforme en $(0, 1)$, su complemento $(1 - U)$ también lo es. Si U toma valores como $0.99, 0.01, 0.5$, $(1 - U)$ tomará $0.01, 0.99, 0.5$. Estadísticamente son idénticos. Por lo tanto, los simuladores simplifican la fórmula para ahorrar una resta:

$$x = -\beta \ln(U)$$

Aplicación: Si el tiempo promedio de juego es $\beta = 20$ minutos y el generador nos da $U = 0.5$:

$$x = -20 \ln(0.5) \approx -20(-0.693) = 13.86 \text{ minutos.}$$

6.4. Ejemplo 2: Distribución Uniforme General (Rangos de Apuesta)

A veces el casino quiere simular una apuesta que no es de 0 a 1, sino que varía uniformemente entre un mínimo A y un máximo B (ej. entre \$10 y \$50).

Paso 1: La Acumulada. Para una uniforme entre $[A, B]$, la probabilidad crece linealmente:

$$F(x) = \frac{x - A}{B - A}$$

Paso 2: Igualar a U.

$$U = \frac{x - A}{B - A}$$

Paso 3: Despejar x. Multiplicamos por el denominador:

$$U(B - A) = x - A$$

Sumamos A:

$$x = A + U(B - A)$$

Interpretación Intuitiva: Esta fórmula es muy lógica:

- Empieza en el mínimo A.
- Toma el ancho del intervalo $(B - A)$.
- Escala ese ancho por el porcentaje U entregado por el generador.

Si $U = 0.0$, obtenemos A. Si $U = 1.0$, obtenemos B. Cualquier valor intermedio es una interpolación proporcional.

6.5. Resumen de la Sección

El método de la transformada inversa nos da un poder inmenso: si podemos integrar la densidad de probabilidad para obtener una fórmula cerrada de $F(x)$ y podemos despejar algebraicamente x, podemos simular **cualquier** comportamiento del mundo real usando solo la función simple `random()` de la computadora.

Referencias

[1] Law, A. M. (2015). *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill. Cap. 8.

[2] Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley. Cap. 3.

[3] Ross, S. M. (2006). *Simulation*, 4th ed. Academic Press. Cap. 5.

[4] Banks, J., Carson, J., Nelson, B., & Nicol, D. (2014). *Discrete-Event System Simulation*, 5th ed. Pearson. Cap. 8.